「意得则舒怀以命笔　理伏则投笔以卷怀」

Zhimu Yang

优化方法导引（Optimization）

优化方法导引（**Optimization**）

一、引言

1. 什么是优化
2. 怎么进行优化

二、建模

1. 优化模型
   - I. 问题描述
   - II. 目标函数
   - III. 数据

三、求解

1. 单目标优化
   - I. 数值优化
     - i. 一阶优化
       - 梯度下降法（Gradient Descent）
       - 最速梯度下降法（Steepest Descent Method）
       - 批量梯度下降（BGD）
       - 随机梯度下降（SGD）
       - 小批量梯度下降（MBGD）
     - ii. 二阶优化
       - 牛顿法（Newton Method）
       - 拟牛顿法（Quasi-Newton Method）
     - iii. 分治
       - 坐标下降法（Coordinate Descent）
       - SMO算法
   - II. 解析优化
     - i. 等式约束问题
       - 拉格朗日乘数法（Lagrange Multiplier Method）
     - ii. 等式及不等式约束问题
       - KKT条件
2. 多目标优化
   - 标量化法
   - ϵ 约束法
3. 特殊情况的优化
   - 凸优化

> *Type:*
>
> *Tutorial / ~~Note~~*

> *For:*
>
> *Amateurs / ~~Professionals~~*
>
> *Reference:*
>
> *[1] 雷明. 《机器学习的数学》. 北京：人民邮电出版社, 2021.1*
>
> *图例及例题部分源自雷明老师，因其例证优秀，故不做删改以引用，图为本人重新绘制。*

一、引言

1. 什么是优化

回想一些你熟悉的场景或是题目——

炎炎夏日，冰棍店的老板想要进一批冰棍，进货价每根 x 元，售价每根 y 元，销量随气温变化，那么下一次进货多少根期望的利润才能最大呢？

一家商场年末打算促销，折扣越多，销量也会越高，但是相应的每件商品得到的利润也会变少，根据已有的销量与折扣的数据，那么折扣应定为多少才能使得利润最大呢？

一批 n 斤的货物将从A地运往B地，采用陆运每次能运 x 斤，需要 a 小时，采用水运每次能运 y 斤，需要 b 小时，怎么选择运货方案使得总共用时最短呢？

某地区楼房的价格在持续上升，价格随月份有部分数据，预测下一个月份的期望楼房价格。

……

这些熟悉的问题都无一例外地有着固定的描述——寻找期望的方案（也即是寻找固定范围内的最大或最小值），并据此逐步改进现有方案，谓之“优化”。本书中我们将会依照个人的理解与教学需要去讨论与介绍一些简单易懂的应用于人工智能的优化方法，而非依照传统的运筹学框架。

2. 怎么进行优化

我们来回顾一些高中时进行优化的例子。

例1. 炎炎夏日，冰棍店的老板想要进一批冰棍，进货价每根 1元，售价每根 3 元，销量一般在50~100之间，那么下一次进货多少根期望的利润才能最大呢？

这个例子很容易理解。目标是使得利润最大，我们记利润为 w ，销量为 n ，得到利润的表达式如下：

$$w=(3-1)\cdot n，50\leq n\leq 100$$

显然，当销量 n 取100的时候，根据利润的表达式，取得最大值，利润最大为200。

例2. 以下是某地区2018-2022年的人均收入情况，请预测2023年期望的人均收入。

年份	2018	2019	2020	2021	2022
年份代码 x	1	2	3	4	5
人均收入 y	1.0	1.2	1.3	1.5	1.6

或许还记得有如下的线性最小二乘公式：

$$\hat{b}=\frac{\sum_{i=1}^{n}x_i\cdot y_i-n\cdot\overline{x}\cdot\overline{y}}{\sum_{i=1}^{n}x_i^2-n\cdot\overline{x}^2}=\frac{\sum_{i=1}^{n}(x_i-\overline{x})\cdot(y_i-\overline{y})}{\sum_{i=1}^{n}(x_i-\overline{x})^2}$$

我们通过线性最小二乘公式计算线性回归方程：

$$\hat{y}=\hat{b}x+\widehat{a}$$

通过求得期望的系数，得到期望的方程，计算出目标预测值，我们这里仍然认为是一种优化。

我们对上述的问题做些总结：

我们建立数学模型，通过数学表达式来描述一个优化问题，进而优化问题就变成了一个求某个函数中理想值（最大值也好最小值也好适中值也好）的数学问题，剩下的就是如何求一个函数的某个特殊值。

二、建模

1. 优化模型

I. 问题描述

如同我们上一章介绍，先建模转化成数学表达式，然后求表达式最值。我们接下来手把手建立一个优化的数学模型。

我们把优化的模型表述成这样的形式：

$$\min_{x} f(x)$$

即求使得 f(x) 表达式取得最小值的 x ：

求使得利润最大的进货量；

求使得利润最大的折扣；

求使得用时最短的运输方案；

求使得楼房预测价最贴近楼房真实价的回归方程；

……

这些例子均能由上述优化模型的表达形式描述不是吗？可能你会好奇——为什么一定是 min ？只通过 min 能够描述所以的问题吗？事实上，哪怕是最大值问题，当你为它添上一个负号后，不就都能由最小值 min 描述了嘛，即使是最优值的问题，也可以视为是在一定约束条件下的最小值问题。

当然，我们可不能忘记了取值范围，某些值仅能在范围内取到，或者在范围内才有极值。所以我们给该问题加上范围：

$$\min_{x} f(x), \quad s.t.\ x \in K$$

s.t. 代表 subject to ，表示受限于。K 称为优化问题的**可行域（Feasible Region）**，也就是该优化问题只限于当 x 位于范围 K 内时才成立。当然，限制条件有时不仅仅是一个范围，也许是一个函数，或是一个等式。而在该范围内求解出的满足条件的解称为**可行解（Feasible Solution）**。上式就是优化模型的模型表述。

好奇的你一定会发现，光从解析的角度看，目标函数 f(x) 会不会存在无法直接求解的情况？是的。同时，优化问题的模型描述并不仅是单纯要求我们去计直接计算得到最优目标，我们仍然可以采用迭代等方法去逼近目标。就解决这个优化问题，分为了**解析优化（Analytical Optimization）**及**数值优化（Numerical Optimization）**。解析优化针对于可以求解的问题，根据解析表达式一步直接求目标值；数值优化针对于无法直接求解的问题，从极限的角度，不断迭代逐步优化使得数值最终无限接近于目标值。

总结一句话即是，对于这样的优化模型，不一定要直接求解，我们可以不择手段地去获得它的目标值。

II. 目标函数

很明显，在优化模型的描述中，首先要确定**目标函数（Objective Function）** $f(x)$，在机器学习、深度学习中我们也称为**损失函数（Loss Function）**。优化的问题纷繁复杂，我们认为把目标转换为成数学形式大概就是两种思路：

（1）直接描述类型——能够直接写出解析式的目标函数。

如例1，能够直接写出利润的表达式：

$$w = (3-1)\cdot n，50 \le n \le 100$$

对于（1）直接描述类型，我想我们不必介绍，根据题意建立即可。

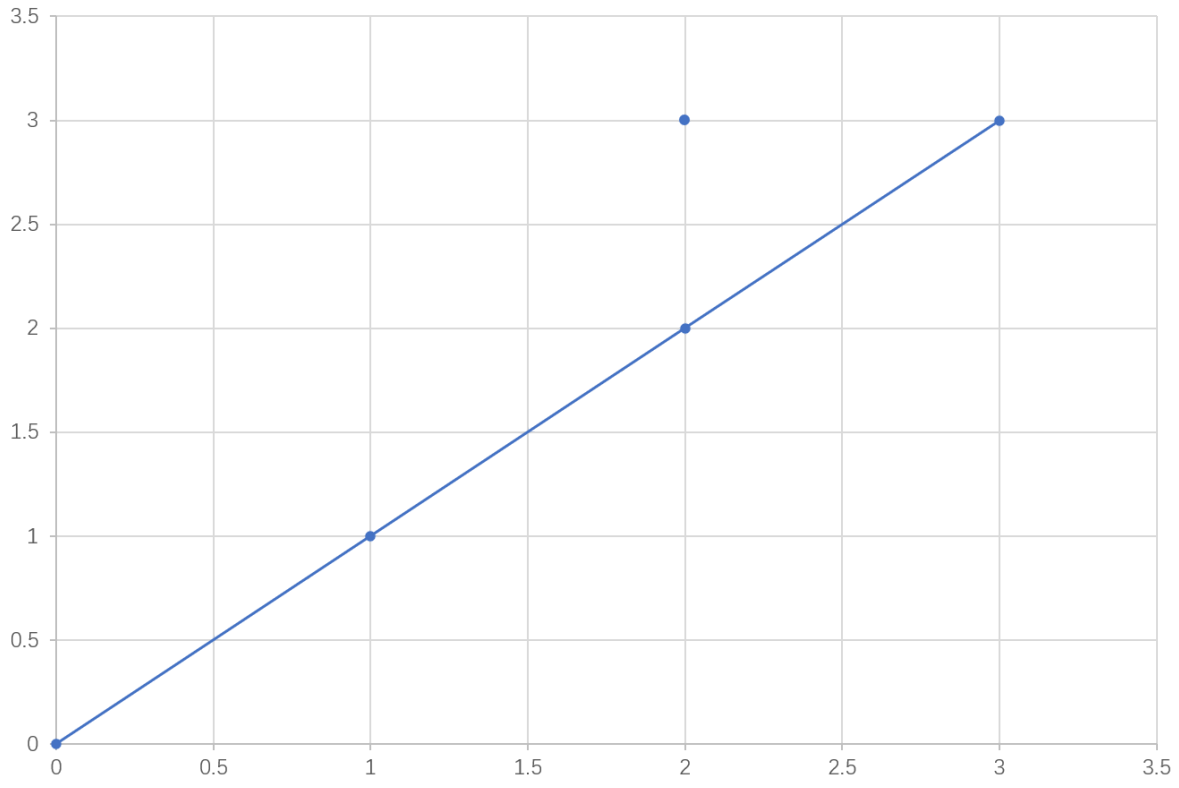
（2）间接描述类型——通过差值等评价指标构建目标函数，使得评价指标最优。

如例2，不能直接写出预测值的表达式，通过线性最小二乘公式计算出系数后方能构建收入的线性回归表达式：

$$\hat{y} = \hat{b}x + \widehat{a}$$

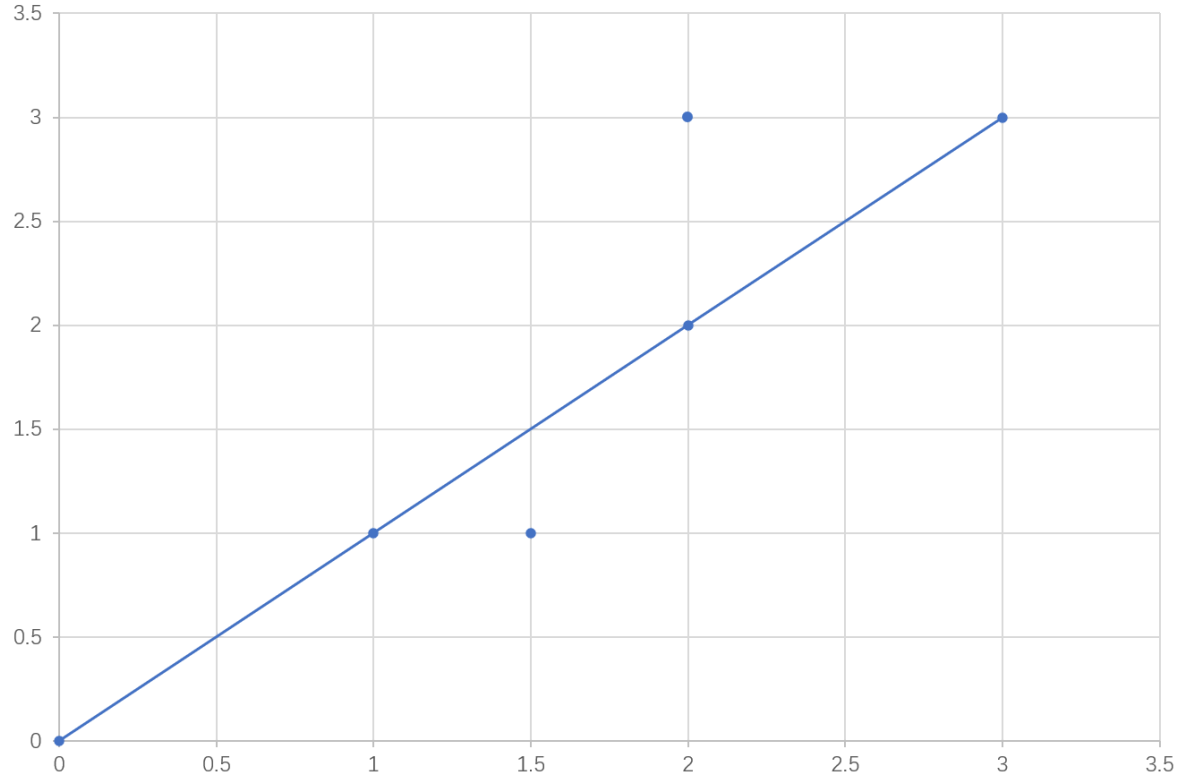
对于（2）间接描述类型，我们将从最小二乘法的推导开始，一步步讨论如何引入间接的评价指标来衡量优劣。

现在我们有这样一副图：

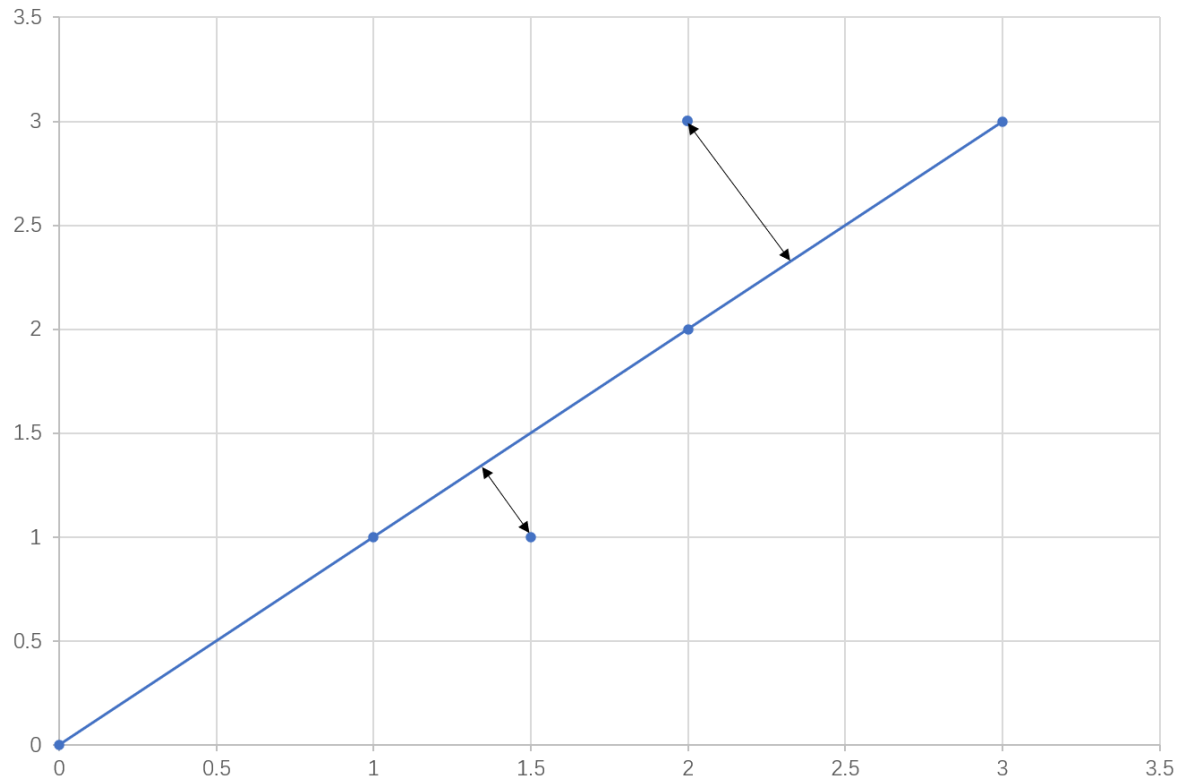


考虑这样的优化问题，我们怎样求解最优的直线，使得它最适合图中上方的(2, 3)点呢？理所当然是穿过该点了！这样的优化问题具有确定的解析表达式。

我们稍微给问题加点难度：



现在求解最优的直线，使得它尽可能地适合图中的两点$(1.5, 1)$与$(2, 3)$。这似乎难以直接以优化模型直接描述，所以我们通过一些其他评价指标去表述它——距离！我们只要使得我们求解的最优直线到两点的距离之和最短，是否就可以认为是已经尽可能地“适合”了呢？不妨一试，我们将垂直直线的长度视为距离：



对于直线：

$$y = ax + b$$
$$ax - y + b = 0$$

与已知点：

$$A(x_1, y_1) \quad B(x_2, y_2)$$

现在我们根据点到直线的距离公式，写出这两点到直线的距离之和的表达式：

$$\begin{aligned} d &= d_1 + d_2 \\ &= \frac{|ax_1 - y_1 + b|}{\sqrt{a^2 + (-1)^2}} + \frac{|ax_2 - y_2 + b|}{\sqrt{a^2 + (-1)^2}} \\ &= \frac{|ax_1 - y_1 + b| + |ax_2 - y_2 + b|}{\sqrt{a^2}} \end{aligned}$$

最终求解最优直线的问题就可以被表达成这样的形式：

$$\min_{a,b} d$$

$$\min_{a,b} \frac{|ax_1 - y_1 + b| + |ax_2 - y_2 + b|}{\sqrt{a^2}}$$

即求解使得距离 d 最小的 a, b 的值。问题从求解一条直线，被转变成了求解直线的参数，而可以被视为一个优化问题。

距离（Distance）

距离，便是最为普适与理想的评价指标。对于绝大多数的问题，如果无法直接求解解析式，那么以距离作为中间评价指标来构建目标函数，转化为优化问题的方法是相当可行的。但是，我们需要对“距离”这个概念有一些思想上的转变——什么是“距离”？

对于一维数轴，我们可以是否可以认为绝对值是一种距离：

$$d = |x_1 - x_2|$$

对于二维平面，我们又可以有欧式距离的表达：

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

以此为准，我们归纳“距离”的概念：

1）需要是非负的。

2）需要是对称的，即从A点到B点的距离等于从B点到A点的距离。

3）需要符合三角不等式，即两边之和大于第三边。

距离的意义在于，可以将多个点映射为函数从而方便构建模型。无论最终是点到线、点到面、线到线等，终归是在求解点到点的问题，所以采用距离作为评价指标进行映射是绝佳的。同时，我们需要认识到，“距离”不仅仅是距离，在稍后的章节中，会出现——差的绝对值、平方差等等各式各样的“距离”，其本质都是距离。对于价格也好分数也好，一旦画到坐

标系中，就可以根据距离衡量这些数据值之间的差异，这点是我们需要明确的。

III. 数据

数据是优化不可或缺的一部分，我们常常依赖以往的数据来改进现在、预测未来。

年份	2018	2019	2020	2021	2022
年份代码 x	1	2	3	4	5
人均收入 y	1.0	1.2	1.3	1.5	1.6

我们重新审视这张在前面出现过的表格，并依此介绍“数据”。先前我们提到过，该表格的数据符合线性回归方程：

$$y = ax + b$$

其中的 x 与 y 即为表中的年份代码 x 与人均收入 y 。那么便有了类似的描述：人均收入 = a * 年份代码 + b 。这意味着，我们认为人均收入与年份代码有关，只要给年份代码一个系数，再加上一个常数，便可得到人均收入。

我们将上述描述拓展一下，假设一个情景——一个人美丽程度与她的眼睛、鼻子、嘴巴、身高线性相关，那么我们就有这样的描述：美丽程度 = a * 眼睛 + b * 鼻子 + c * 嘴巴 + d * 身高，并可用这样的线性方程表述：

$$p = aw + bx + cy + dz$$

眼睛、鼻子、嘴巴、身高等都是一个人的特征，同样的，我们通常把这些表征数据特色的自变量称为**特征（Feature）**，特征的数量则为**维度（Dimension）**，而特征构成的函数的值称为**标签（Label）**，因为诸如“美丽程度”等难以准确描述，故将其用标签值量化，这些常常是针对已有数据的。对于上表2023年或以后的未知数据，我们常称其因变量的值为**预测值（Predict）**，这与已知的标签是不同的。

如果在预测人均收入的问题中，表征人均收入的特征变成其余多个特征，有如下表格：

年份	2018	2019	2020	2021	2022
工龄 x_1	1	2	3	4	5
家庭人口 x_2	3	4	4	5	6
学历层次代码 x_3	1	2	4	5	6
人均收入 y	1.0	1.2	1.3	1.5	1.6

无论如何我们仍然假设这是符合线性的，那么就能够得到如下方程：

$$y = ax_1 + bx_2 + cx_3 + d$$

在往后的问题中，特征将会达到非常高的数量级，因而我们并不喜好上式的这种冗长表述，所以我们利用线性代数的知识，将上式简化：

$$y = \vec{w} \cdot \vec{x} + d$$

其中：

$$\vec{w} = (a, b, c) \quad \vec{x} = (x_1, x_2, x_3)$$

向量 x 即为特征，其维度对应特征维度，向量 w 被我们称为权重，其乘积为向量的点积。当然也可以将其视为矩阵相乘（事实上在神经网络中我们就是这么做的）：

$$y = \mathbf{w} \cdot \mathbf{x}^T$$

三、求解

在前面的部分，我们简要的介绍了一遍优化的模型，并粗浅地介绍了如何构建目标函数，在完成了这一切之后，剩下的问题便是——如何求解这个模型中我们期望的值，之后所有的方法遵循着我们先前提到的准则——不择手段。

1. 单目标优化

在这里我们首先区分出单目标优化与多目标优化，这非常容易理解，需要求利润最大是单目标优化，需要求利润最大的同时销售量最小，这就是多目标优化。即便是处理多目标优化，我们常考虑的方法也是转化为单目标优化，接下来就介绍单目标优化的方法。

I. 数值优化

我们依然从数值逼近的角度先开始。数值优化的核心是**导数（Derivative）**及导数的推广——**梯度（Gradient）**，我们这里介绍的方法也都是基于导数或梯度的**迭代**方法。数值优化的本质就是对无法直接求解的函数进行迭代逼近。数值优化的核心在于设计迭代规则，如何从该点向目标点移动。

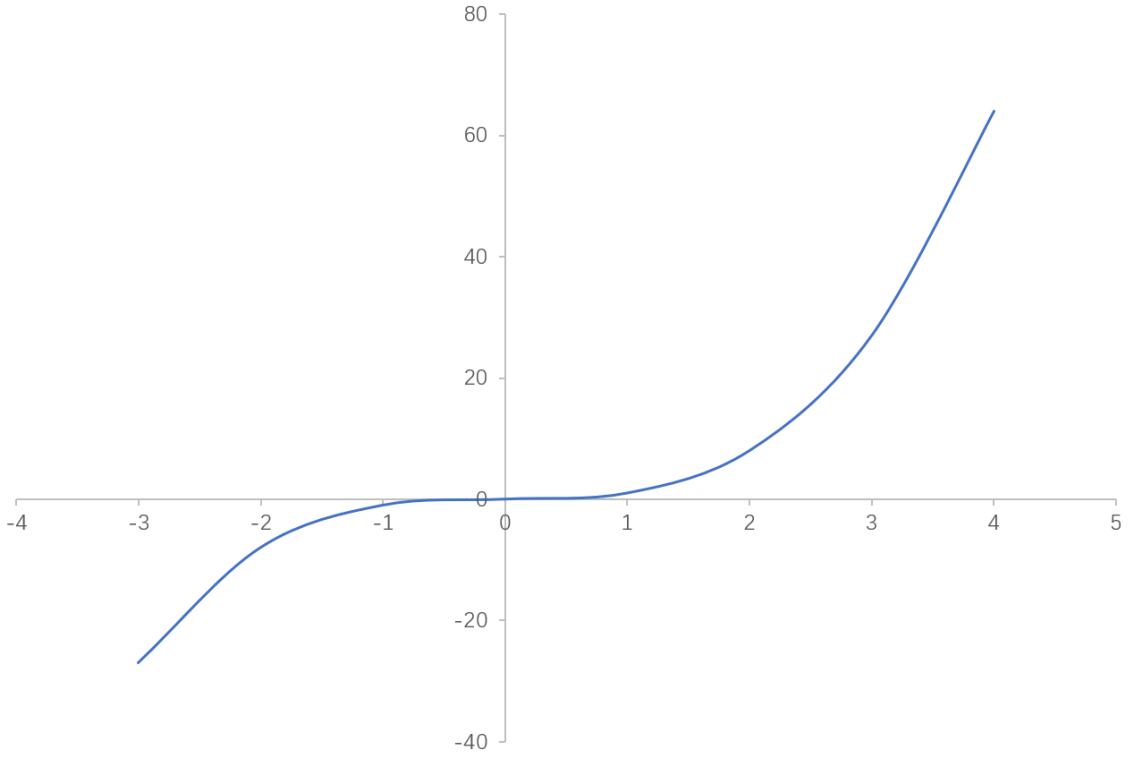
为什么使用导数？

导数的几何意义是位于该点处的斜率，基于这一点我们很容易讨论极值问题。

Fermat's Lemma

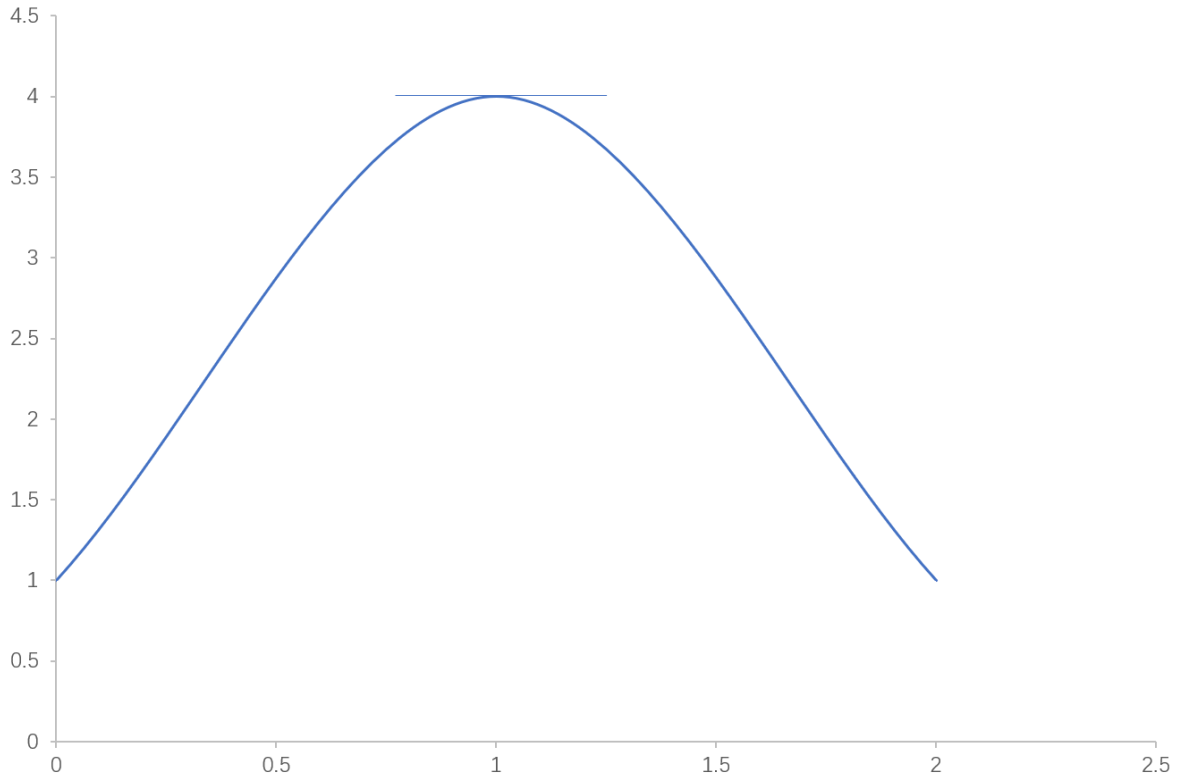
> *设 f (x) 在点 a 的某邻域内有定义，并在 a 处可导，且对邻域内任意 x 都有 f(x) ≤ f(a)(或 f(x) ≥ f(a))，那么 f '(a) = 0.*

费马引理的描述表明，可导函数极值点处的导数必为0，这一描述是不可反过来的，即可导点处取到极值的必要条件为导数为 0 。对于导数为 0 的点，我们称为**驻点（Stationary Point）**，**极值点（Extreme Point）**不仅仅在驻点处取得，也可能在不可导点处取得，下图展示的即是驻点而非极值点的点：



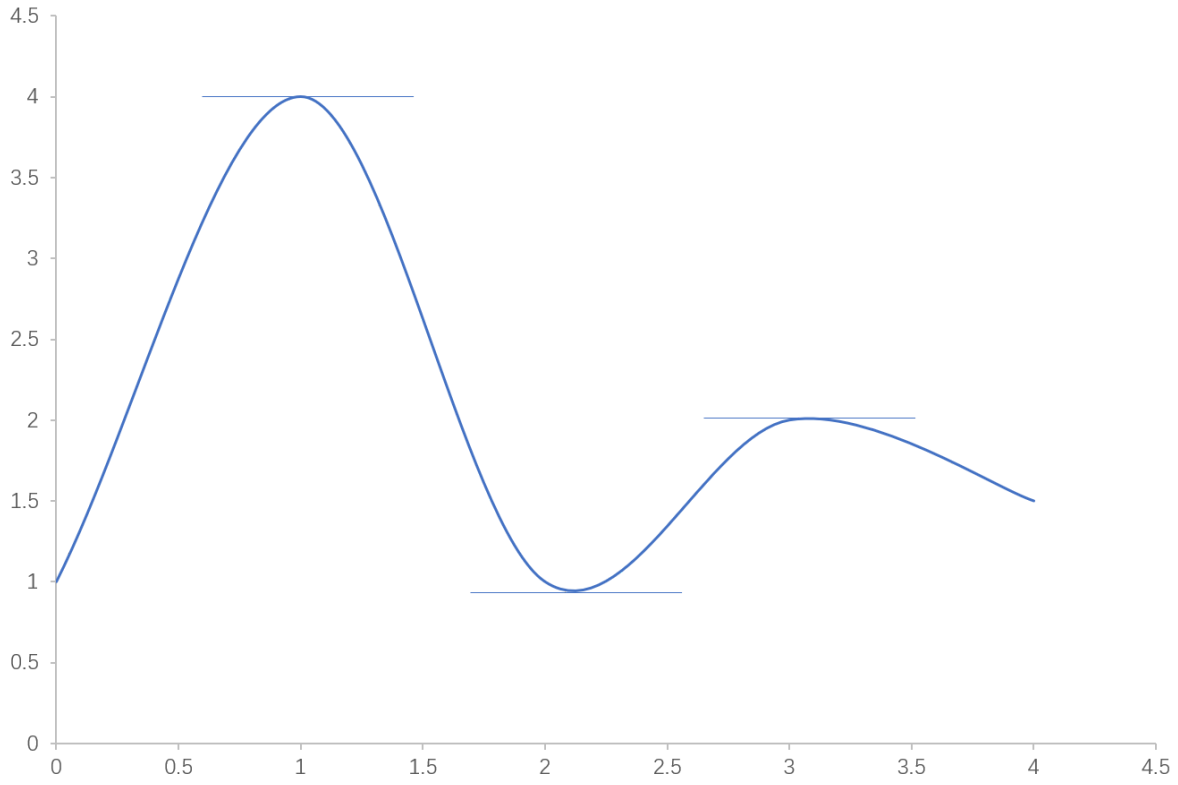
图中的点$(0,0)$是驻点，但非极值点。

费马引理使得我们能够将求优化极大极小值的问题转化为求解导数为0的问题。这在下面的图像中体现的淋漓尽致：



显然，图中的极值点处导数为0，求解导数为0的方程能够得到该点。

但是对于复杂函数，情况又会稍有不同：



当试图对这样的函数求导数为0的点时，我们求得了三个解，对应三个极值点，我们称之为**局部极值点（Local Extreme Point）**，其点处取得的值为**局部极值（Local Extremum）**，因为它们仅在一定邻域中取得极值。对于局部极值点产生的局部最优解 x*，**只在该局部最优解的某邻域内**，我们显然可以得到：

$$f(x^*) \leq f(x) \quad 或 \quad f(x^*) \geq f(x)$$

而所有局部极值点中的最大/最小点，我们称之为**全局极值点（Global Extreme Point）**，其点处取得的值为**全局极值（Global Extremum）**。**在整个优化问题的约束范围内**，我们都可以得到：

$$f(x^*) \leq f(x) \quad 或 \quad f(x^*) \geq f(x)$$

无论对于局部极值还是全局极值，当上式不取等号时：

$$f(x^*) < f(x) \quad 或 \quad f(x^*) > f(x)$$

我们称其是**严格（Strict）**的，对应点即是严格极大/小值点。

显然，求解优化问题需要我们针对有限范围求出全局极值点。在上图的问题中，也就是需要我们将求解出的极值点带入进行检验，寻找最大/最小值。

现实中，我们很难有机会去解仅仅涉及一元函数的问题，对于多元函数，我们需要的是求解梯度为0的点作为极值点，这是导数的推广，也十分容易理解。

在高等数学中我们已经学习过梯度的概念，这里我们只做简单的复习：

斜率——一元函数的导数，梯度——多元函数的偏导数，也就是多元函数在每个坐标轴方向的斜率的分量。

$$\nabla f = (\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z})$$

导数与梯度提供给了我们一个全新的思考方式——在面对一些不知道求根公式的目标函数时，我们可以通过求解导数为0或梯度为0的点直接解出极值点！比如如下的函数：

例. 求该函数的最小值点

$$f(x) = \frac{1}{3}x^3 - x^2 + x + 4$$

解： 直接求解导数为0的点，然后判断是否为最小值：

$$f'(x) = x^2 - 2x + 1$$
$$x = \pm 1$$
$$f(1) = \frac{13}{3}$$
$$f(-1) = \frac{5}{3}$$

显然，在 x=1 处取得最小值，最小值为$\frac{5}{3}$。

我们再尝试一下二元函数：

例. 求该函数的最小值点

$$f(x, y) = x^2 + 2y^2 - 3xy + 4y$$

解： 直接求解偏导数为0的方程：

$$\begin{cases} f_x(x, y) = 2x - 3y = 0 \\ f_y(x, y) = 4y - 3x + 4 = 0 \end{cases}$$

$$\begin{cases} x = 12 \\ y = 8 \end{cases}$$

显然，在$(12, 8)$处取得最小值，最小值为16。

是不是非常得心应手了？不妨试试下面的函数：

例. 求该函数的最小值点

$$f(x, y) = x^3 + 4x + e^{xy} + sin(xy) - y^3$$

解： 直接求解偏导数为0的方程：

$$\begin{cases} f_x(x,y) = 3x^2 + 4 + ye^{xy} + ysin(xy) = 0 \\ f_y(x,y) = xe^{xy} + xsin(xy) - 3y^2 = 0 \end{cases}$$

算着算着一看，不对劲，这样的方程似乎没有公式解，称为**超越方程（Transcendental Equation）**， 或者至少是不好求解，我们无法得到一个确切的解的表达式，即我们无法进行解析优化，这时候我们通过迭代的方法进行数值优化。

i. 一阶优化

一阶优化的方法即是利用一阶导数进行迭代。

梯度下降法（Gradient Descent）

如何利用导数进行迭代？

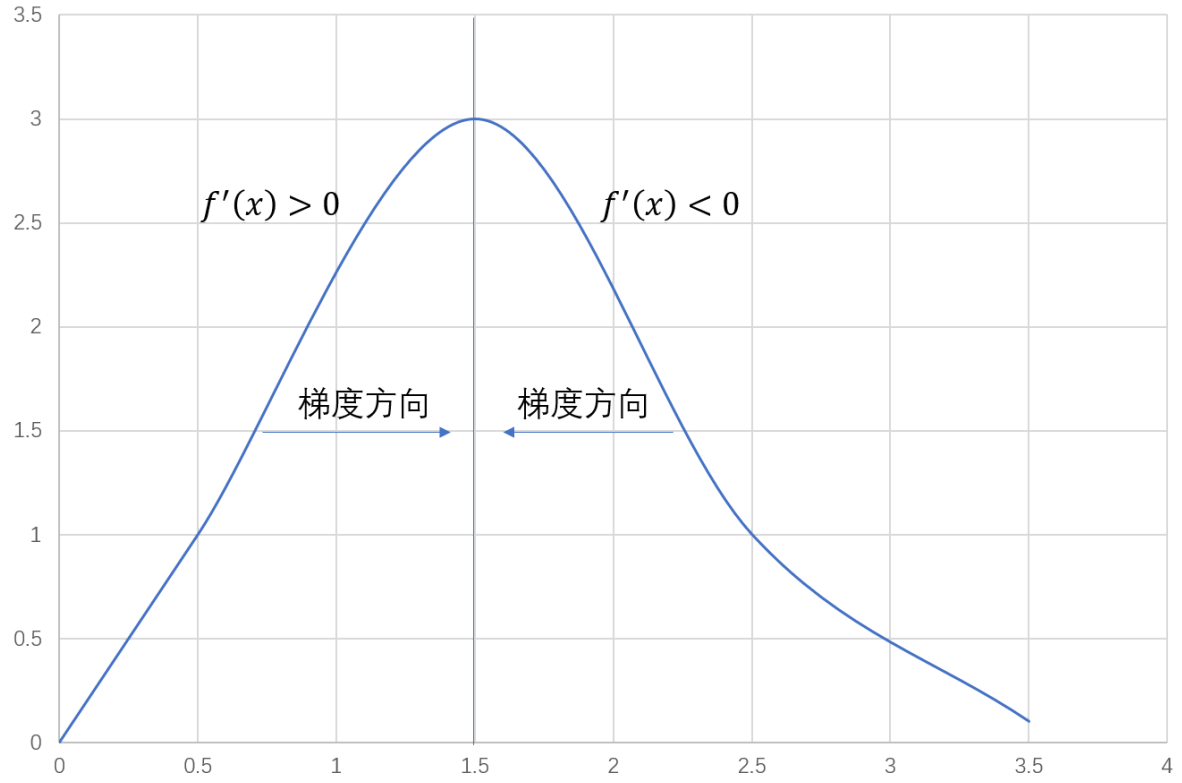
我们上文提到梯度，我们再回顾一下书中对梯度的描述：

> *梯度，表示某一函数在该点处的方向导数沿着该方向取得最大值，即函数在该点处沿着该方向（此梯度的方向）变化最快，变化率最大（为该梯度的模）。*

我们提取出其中的关键——“变化率最快”。这就好比我们下坡，笔直的冲下去能够到达坡底并且最快。这就是梯度下降法，**每走一步计算一下坡度最陡的方向，然后继续向最陡的方向走下一步，最终走到坡底（全局极值点）**。

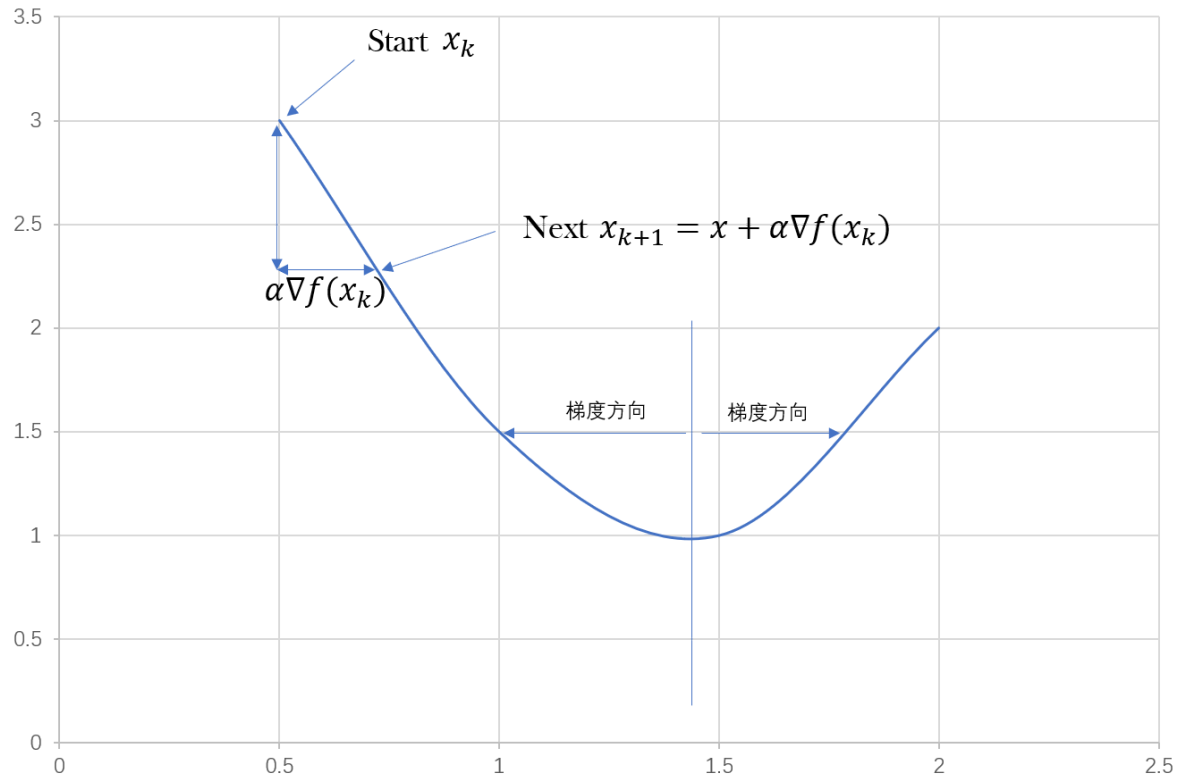
我们先在二维的坐标系中感受一下这一几乎贯穿我们整个专业的方法。

根据梯度的定义，因为在二维坐标系中我们只有 x 一个变量，所以对一元函数的方向导数实际上就只有沿 x 轴这唯一一个方向，即对 x 的导数，这就是说，在二维坐标系中，梯度方向只有两个，x 轴正方向或 x 轴负方向，导数大于0时，梯度方向为 x 轴正向；导数小于0时，梯度方向为 x 轴负向。



虽然名字叫做梯度下降，但是根据目标的不同，也可能是一种上升。在上图中，我们如果需要求最小值，只需沿着负梯度方向一步一步迭代寻找即可，如果需要求最大值，只需沿着正梯度方向一步一步迭代寻找即可，那么一起来试一试吧！

如下图，我们要使用梯度下降求出最小值点。



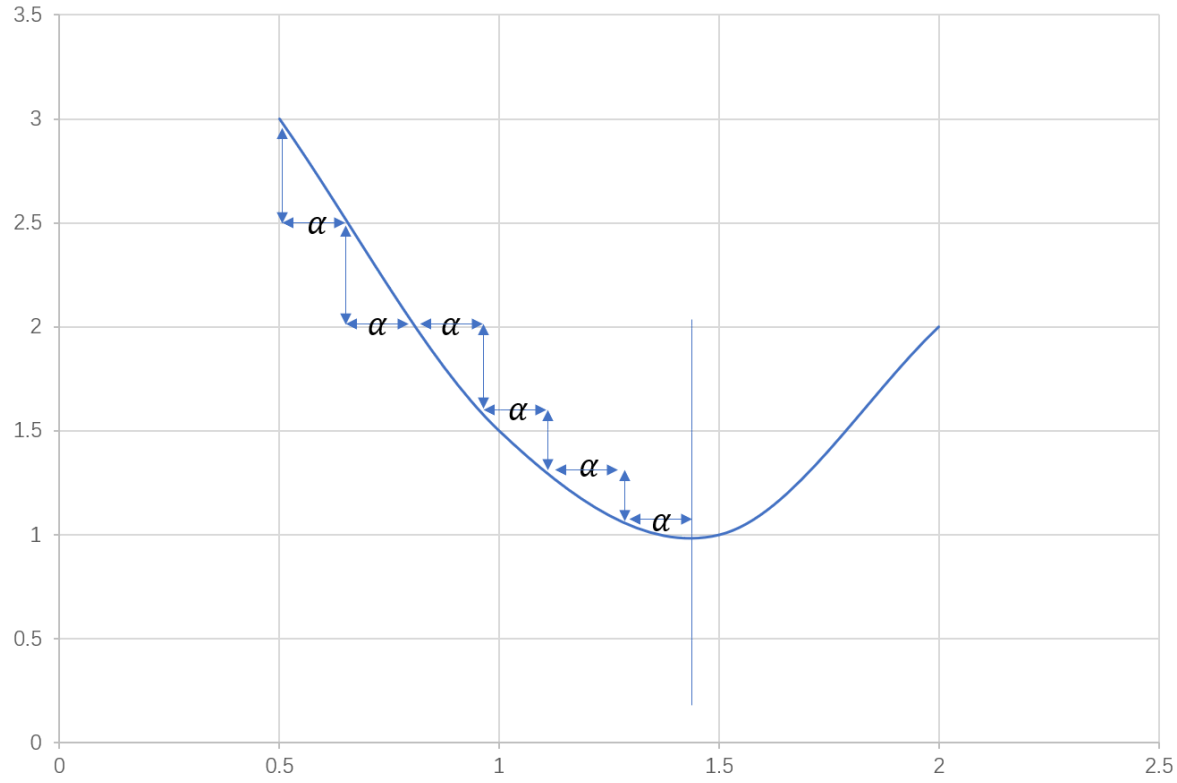
1)首先自然是选取一个点作为我们的开始，就选最左边的那个点吧，我们记这个初始点为 (x_1, y_1) 。

2)计算一下梯度$\nabla f(x)$，导数为负，梯度方向沿 x 轴负向，因为我们要求最小值，那么就选取负梯度方向（即 x 轴正向）一步一步迭代。

3)因为这是迭代的方法，我们不能一步跨太大，否则会直接冲过了最小值点而忽略了它，所以我们选取一个合适的**步长** α，在机器学习等领域中也称为**学习率（Learning Rate）**，我们每次下降只下降这么一小步，我们每次走的长度应为$\alpha \cdot \nabla f(x)$。

4)继而前往的下一个点则为$(x_2, y_2) = (x_1 - \alpha \cdot \nabla f(x), y_2)$。

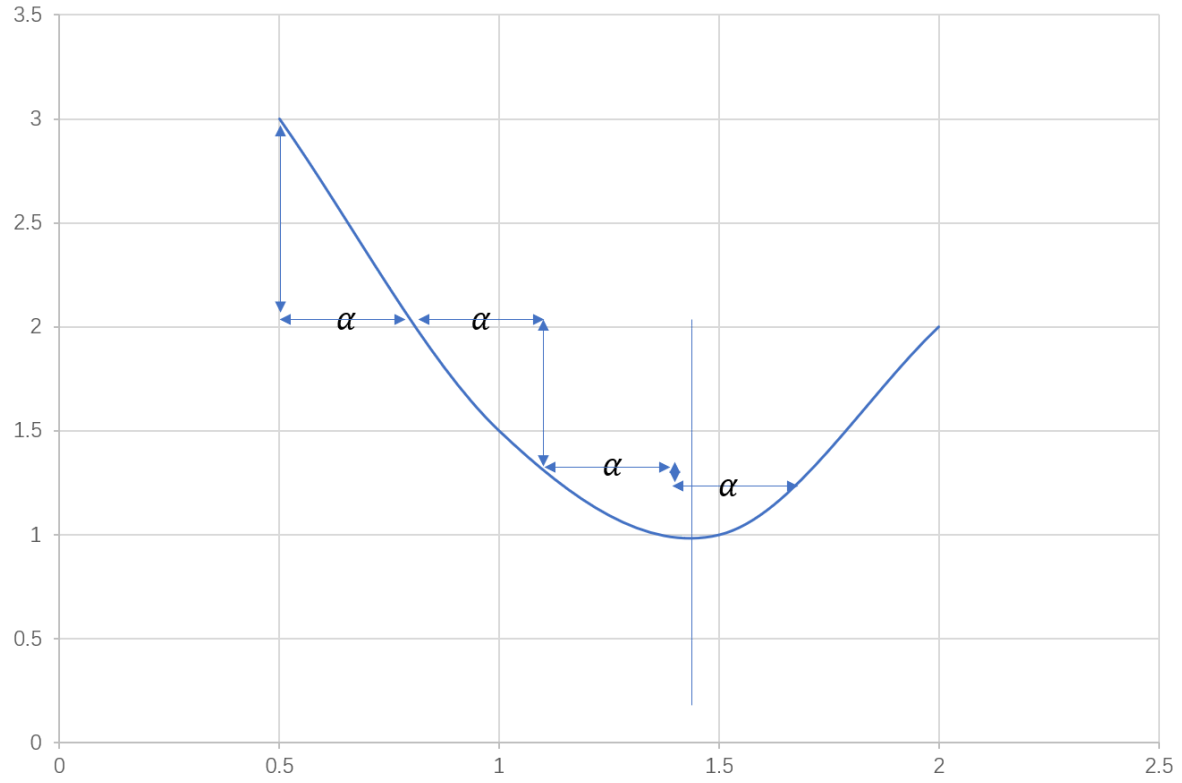
5)重复迭代步骤直至找到最小值点，怎么才能认为找到最小值点了呢？这个问题可以有着很多不同的答案。第一种是认为，目前的点和上一个点的函数值距离小于一定程度时，我们就认为找到了最小值，即当$y_n - y_{n-1} < \epsilon$时（ϵ 的选取依据我们精度要求也可以不同），认为点 (x_n, y_n) 是最小值点；第二种 是认为，梯度无限接近于0时，即当 $\nabla f(x) \to 0$ 时，我们就认为找到了最小值。



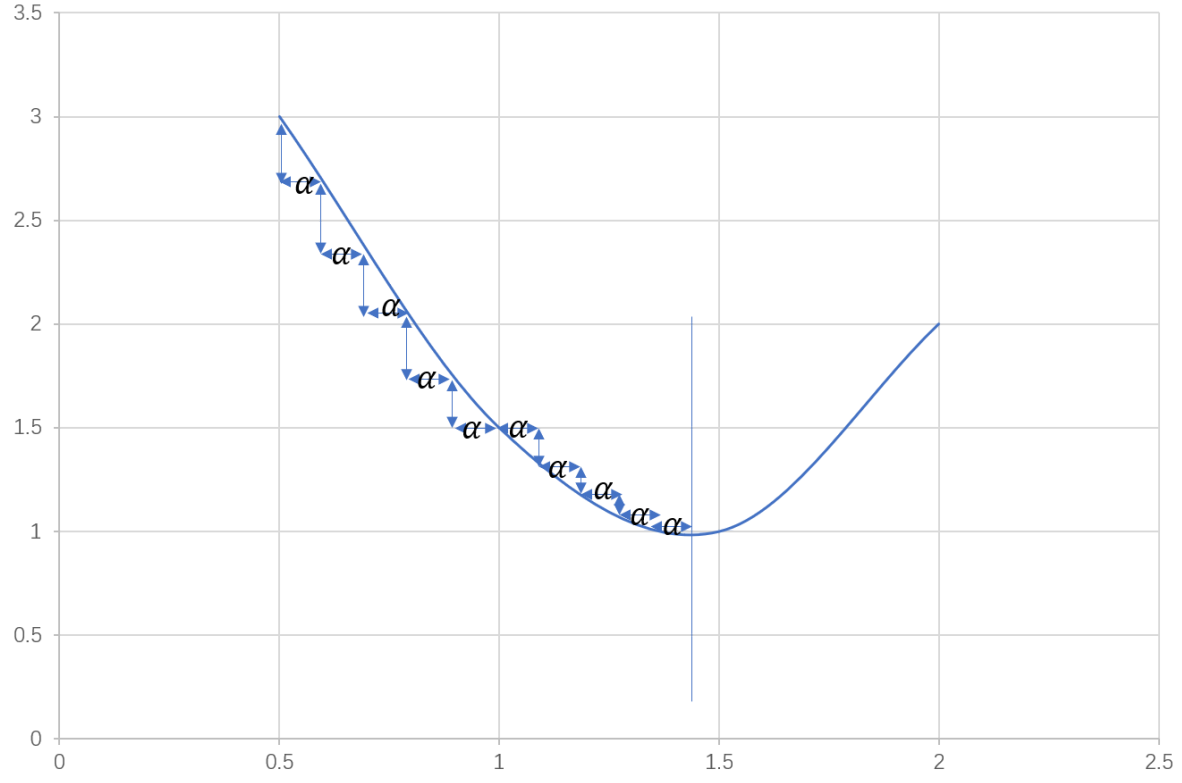
（图中的 α 仅用来说明步长的影响，实际梯度下降中 x 跨越的距离应该是 $\alpha \nabla f(x_0)$）

当步长选取合适时，我们会较快且较准确地逼近最小值点。例如上图，我们将会在6次迭代后逼近最小值点。

当步长选取过大时，下降速度虽然很快，迭代次数也会减少，但是容易错过或达不到最小值点附近。例如下图，即便迭代次数减少到4次，但是无法达到最小值点：



当步长选取过小时，迭代次数增多，结果更容易逼近最小值点，也就是求出的最小值会比较精确，但是下降速度缓慢，耗费计算资源，耗费时间。例如下图，迭代次数徒增到10次，耗时更长：



总结一下梯度下降法的步骤，对于待优化的函数 f：

梯度下降法
1)选取初始点x_0，确定步长α，确定迭代终止条件ϵ

梯度下降法
2)计算当前点的梯度$\nabla f(x_0)$，选取目标梯度方向
3)计算下一点的位置$x_0 - \alpha \cdot \nabla f(x_0)$
4)前往下一个点，进行迭代$x \to x_0 - \alpha \cdot \nabla f(x_0)$
5)重复迭代步骤至$f(x_n) - f(x_{n-1}) < \epsilon$或$\nabla f(x_n) \to 0$，则最小值点为$x_n$

则我们可以得到梯度下降法的算法伪代码：

梯度下降法
x_0=?, k=0
while $\nabla f(x_k) > \epsilon$ and $k < N$:
$x_{k+1} = x_k - \alpha \nabla f(x_k)$
$k = k + 1$

我们使用Python复现梯度下降法计算二元函数$f(x,y) = (x-2)^2 + y^2$的最值点:

```
#初始化
#待求函数
def f(x,y):
    return (x - 2) ** 2 + y ** 2
#待求函数的梯度
def dx(x):
    return 2 * (x - 2)
def dy(y):
    return 2 * y

k = 0
fxy = 0
fxy_last = 1
N = 1000  #迭代上限,防止陷入死循环
epsilon = 0.0001  #用于判定是否达到极值
step = 0.01  #步长
x = 100  #初始点
y = 100  #初始点
```

```
#梯度接近于0时停止
while k < N and dx(x) > epsilon and dy(y) > epsilon:
    x -= step * dx(x)
    y -= step * dy(y)
    fxy = f(x,y)
    k += 1
    print("第%d次: x = %f, y = %f, f = %f, dx = %f, dy = %f" % (k+1,
x, y, fxy, dx(x), dy(y)))
```

```
#当前点和上一点的函数值差值接近于0时停止
while k < N and abs(fxy - fxy_last) > epsilon:
    fxy_last = fxy
    x -= step * dx(x)
    y -= step * dy(y)
    fxy = f(x,y)
    k += 1
    print("第%d次: x = %f, y  =%f, f = %f, dx = %f, dy = %f" % (k+1,
x, y, fxy, dx(x), dy(y)))
```

可能你会质疑——这求导还得自己求，这不是糊弄人吗？我们可以使用sympy库的函数协助我们求导：

```
from sympy import diff
from sympy import symbols

def f(x, y):
    return (x - 2) ** 2 + y ** 2

#调用求导函数前需要先用symbols定义待求函数的变量: x、y、z···
x = symbols('x')
y = symbols('y')

#调用diff函数求导
#diff(f, x, 1) 对函数f求对x的一阶偏导(1阶可以省略)
#diff(f, y, 2) 对函数f求对y的二阶偏导
#diff(f, x).subs(x, 2) 对函数f求对x的一阶偏导，并计算x=2时的值
print(float(diff(f(x, y), x)))
print(float(diff(f(x, y), x).subs(x, 1))
```

大家可以尝试把上文中梯度下降法的简单代码换为使用diff函数求导的代码，当然求导的方法也不止这一种，这里由于作者很懒就不写了。

最速梯度下降法（Steepest Descent Method）

在梯度下降中，我们确定了固定的步长 α ，每次按照固定的步伐前进。有的时候，我们不妨步子迈大一些，可以减少一些不必要的步数。最速梯度下降就是梯度下降，只不过我们**每一步都需要再多计算一个最优步长** α ，然后利用这个最优步长进行下降。

问题的核心就是在于，如何在每一步中计算最优步长 α 。“最优”步长，想必大家又意识到了，求解最优步长同样是一个优化问题，很像是套娃对吧，我们给出求解最优步长优化问题的数学描述：

在 n 点处：

$$\underset{\alpha}{min}\ f(x_n - \alpha\nabla f(x_n))$$

关于如何求解这个优化问题，方法仍然非常之多。这里我们还是只介绍基础的方法——我们通常在这里使用一些线搜索算法求解步长优化问题，从 x_k 点开始，步骤如下：

（1）初始化一个步长 $\alpha_0 = 1$ 和参数 c_1 、β

（2）计算下一点 $x_{k+1} = x_k - \alpha\nabla f(x_k)$

（3）计算$\Delta = \alpha_k c_1 \nabla f(x_k) * (-\nabla f(x_k))$

（4）如果 $f(x_{k+1}) \leq f(x_k) + \Delta$，则返回步长 α_k ，否则，缩小步长 $\alpha_k = \beta\alpha_k$

（5）重复迭代步骤

批量梯度下降（BGD）

接下来我们将梯度下降正式引入人工智能的领域中。相信大家也经常听说，人工智能有一个训练过程。我们把人工智能的优化问题简化成这样的表述：我们有一堆已知的数据 (x_i, y_i) ，但是不知道 x 和 y 之间具体的关系式，我们先初始化一个随机的关系式 $y = wx + d$ ，我们需要用这些已有的数据 (x_i, y_i) ，求出最合适的 w 和 d 。

我们说过，优化方法的本质就是求最小值，我们如何让上面的问题变成一个可以求解最小值的优化问题？对——利用距离或是其他的衡量指标，我们把已知数据的每个点 x_i 带入初始化的随机函数求出一个 $\widehat{y_i}$ ，然后计算这个值与已知数据点的函数值 y_i 间的差值 $L = |\widehat{y_i} - y_i|$，这个差值最小时的参数 w、d 就是最优的（事实上我们可以把 d 当作一种 w 而忽略）。我们稍微把公式推导一下吧。

$$L(w, x_i, y_i) = |\widehat{y_i} - y_i| = |wx_i + d - y_i|$$

注意注意，这只是对其中一个数据点而言的，我们训练时，已知的数据点 (x_i, y_i) 显然有 N 组，所以最终的目标函数/损失函数应该是：

$$L(w) = \frac{1}{N}\sum_{i=1}^{N} L(w, x_i, y_i) = \frac{1}{N}\sum_{i=1}^{N} |wx_i + d - y_i|$$

最终我们可以把优化问题简写为：

$$\min_{w} L(w)$$

有了这样的优化问题描述和精确的待解函数 $L(w)$，想必大家就知道如何把梯度下降法用到其中了吧，这里大家可以推导一下梯度下降法用在该问题中的求解步骤和具体公式。

那么什么是批量梯度下降呢？在对上述问题使用梯度下降时，我们会发现，每一次迭代时，我们都要计算 $\sum_{i=1}^{N}$ 这个求和然后平均，从第1个数据到第N个数据全部都要算上，使用了全部的样本数据（1-N）。每次迭代使用所有样本数据的，就是**批量梯度下降**。

```
import torch

x = torch.tensor([1, 2, 3, 4], dtype=torch.float32)
y = torch.tensor([1, 2, 3, 4], dtype=torch.float32)

w = torch.tensor(0.0, requires_grad=True)
d = torch.tensor(0.0, requires_grad=True)

l = torch.nn.MSELoss()

optimizer = torch.optim.SGD([w, d], lr=0.01)
epochs = 100

for epoch in range(epochs):
    y_pred = w * x + d
    loss = l(y_pred, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

print(f'w = {w.item()}, d = {d.item()}')
```

显然批量梯度下降是非常基础常用的，但是每次都要遍历整个数据集，如果数据集非常大，岂不是非常耗时？

随机梯度下降（SGD）

随机梯度下降就想着，既然每次计算求和很麻烦，好！那我不求这和然后求平均了，我就只用其中一个来求行不行？（$\frac{1}{5}(1+2+3+4+5)$ 计算量很大，我们直接随机取一个值 3 来代表，计算量瞬间下降）我对精度要求没有那么高，不需要多么优的解，差不多就行，就是为加快速度。行，这就是随机梯度下降：

$$L(w) = L(w, x_i, y_i) = |wx_i + d - y_i|$$

```
import torch

x = torch.tensor([1, 2, 3, 4], dtype=torch.float32)
y = torch.tensor([1, 2, 3, 4], dtype=torch.float32)

w = torch.tensor(0.0, requires_grad=True)
d = torch.tensor(0.0, requires_grad=True)

l = torch.nn.MSELoss()
optimizer = torch.optim.SGD([w, d], lr=0.01)

batch_size = 1
epochs = 100

for epoch in range(epochs):
    loader =
torch.utils.data.DataLoader(torch.utils.data.TensorDataset(x, y),
batch_size=batch_size, shuffle=True)
    for x_batch,  y_batch in loader:
        y_pred = w * x_batch + d
        loss = l(y_pred, y_batch)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

print(f'w = {w.item()}, d = {d.item()}')
```

我们当然需要为算法的快速付出一些代价——它尽管每次迭代很快，但是因为没有平均值的指导，它下降的过程有些盲目，迭代次数会增多，也不一定找得到最小值。

小批量梯度下降（MBGD）

做人我们不能太极端，上述的两种梯度下降方法有些太暴力了，不是跟计算量死磕就是跟速度死磕，实际我们使用时，往往是对两者的折中——小批量梯度下降，我们既不用所有样本求平均，也不随机用一个样本计算，而是小批量地求平均（假若有100个样本，我们只用50个求平均），既降低计算量又提高精度。

假设我们样本数据有 N 个，我们取小批量大小为 $m(m < N)$ 来进行训练：

$$L(w) = \frac{1}{m}\sum_{i=1}^{m} L(w, x_i, y_i) = \frac{1}{m}\sum_{i=1}^{m} |wx_i + d - y_i|$$

```
import torch

x = torch.tensor([1, 2, 3, 4], dtype=torch.float32)
y = torch.tensor([1, 2, 3, 4], dtype=torch.float32)

w = torch.tensor(0.0, requires_grad=True)
d = torch.tensor(0.0, requires_grad=True)

l = torch.nn.MSELoss()
optimizer = torch.optim.SGD([w, d], lr=0.01)

batch_size = 5
epochs = 100

for epoch in range(epochs):
    loader =
torch.utils.data.DataLoader(torch.utils.data.TensorDataset(x, y),
batch_size=batch_size, shuffle=True)
    for x_batch,  y_batch in loader:
        y_pred = w * x_batch + d
        loss = l(y_pred, y_batch)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

print(f'w = {w.item()}, d = {d.item()}')
```

ii. 二阶优化

由于梯度下降法收敛速度很慢，我们接下来要尝试利用二阶导数来加快收敛速度。

为什么要使用二阶导数？

对于一个函数，光知道它的表达式，很容易出现无法计算的情况，由此我们使用导数来观察函数的图像走势，方便我们选取方向，而二阶导数为我们提供了进一步的图像凹凸信息，也就是说，**对于一个函数，我们知道的信息越多，越有利于我们选择前往最值点的方式**。

牛顿法（Newton Method）

我们先前提到，数值优化方法的核心是如何从当前点前往下一个点，梯度下降的理解是在迭代点处选择负梯度方向移动，而牛顿法则是**在迭代点处将函数近似为可解函数，然后选择可解函数梯度为0的方向移动**。

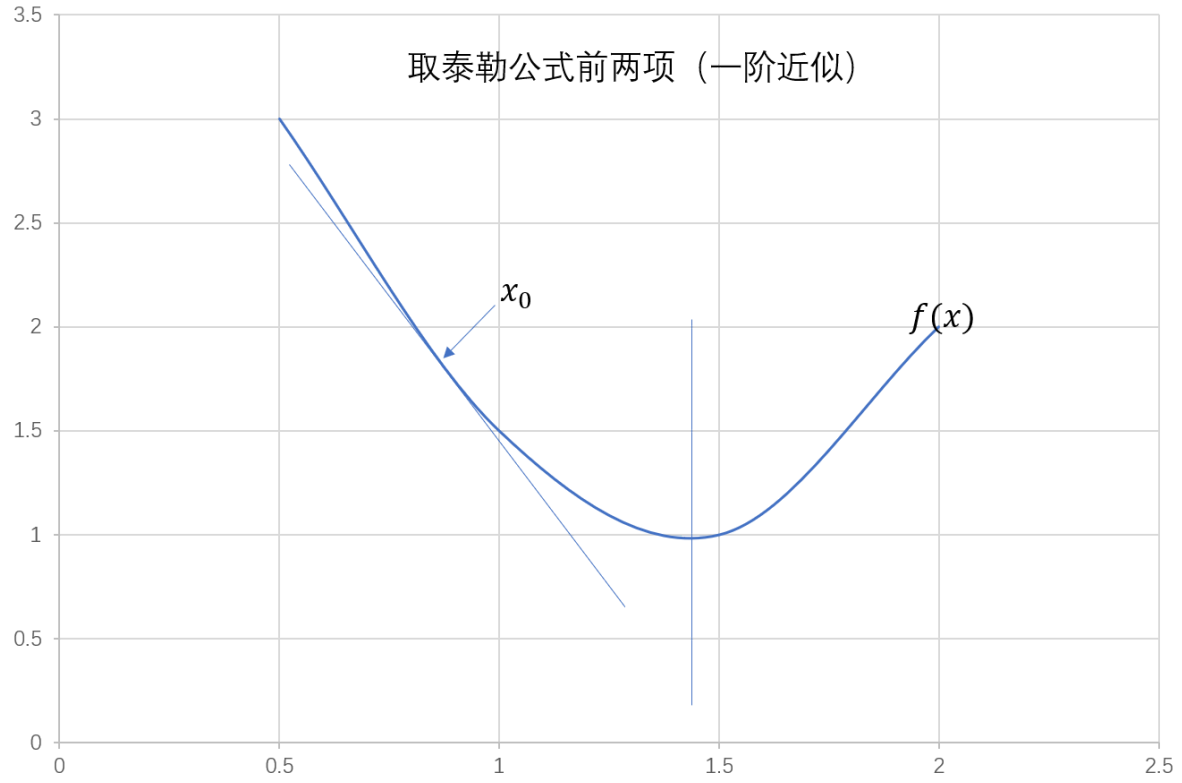
想一想我们在高等数学中学习的知识，怎么将在某一点将一个曲折的无法求解的函数近似为一个可以求解的函数？——泰勒展开。

泰勒公式（Taylor Formula）

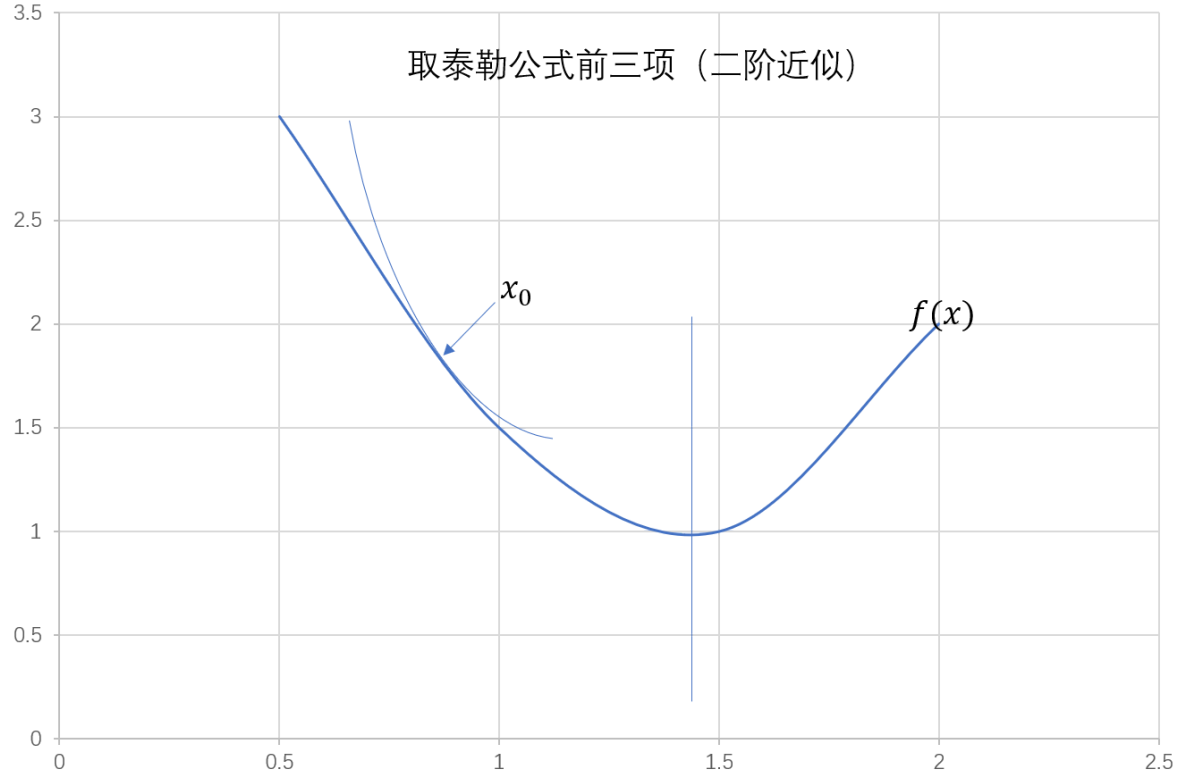
> *邻域内具有n阶导数的函数$f(x)$在x_0处可展开为：*
>
> $$f(x)=f(x_0)+f'(x_0)(x-x_0)+\frac{f''(x_0)}{2!}(x-x_0)^2+\cdots+\frac{f^{(n)}(x_0)}{n!}(x-x_0)^n+R_n$$

此时，我们只需取泰勒公式前几项做近似，既减轻计算量，也能得到很好的近似效果。

一阶牛顿法，也就是取前两项做一阶近似，可以看出我们实际是把函数当作在该点的切线处理的。



二阶牛顿法，也就是取前三项做二阶近似，可以看出我们实际是把函数当作在该点的二次函数处理的。



我们通过更直观的一元函数一阶牛顿法理解该方法，然后推广介绍多元函数二阶牛顿法。

我们从横坐标为x_1的A点出发，寻找函数$f(x)$的极值点：

（1）在A点时，根据牛顿法的介绍，我们对函数在A处进行泰勒展开：

$$f(x) \approx f(x_1) + f'(x_1)(x - x_1)$$
$$y = f(x_1) + f'(x_1)(x - x_1)$$

显然这就是函数$f(x)$在A点的切线。

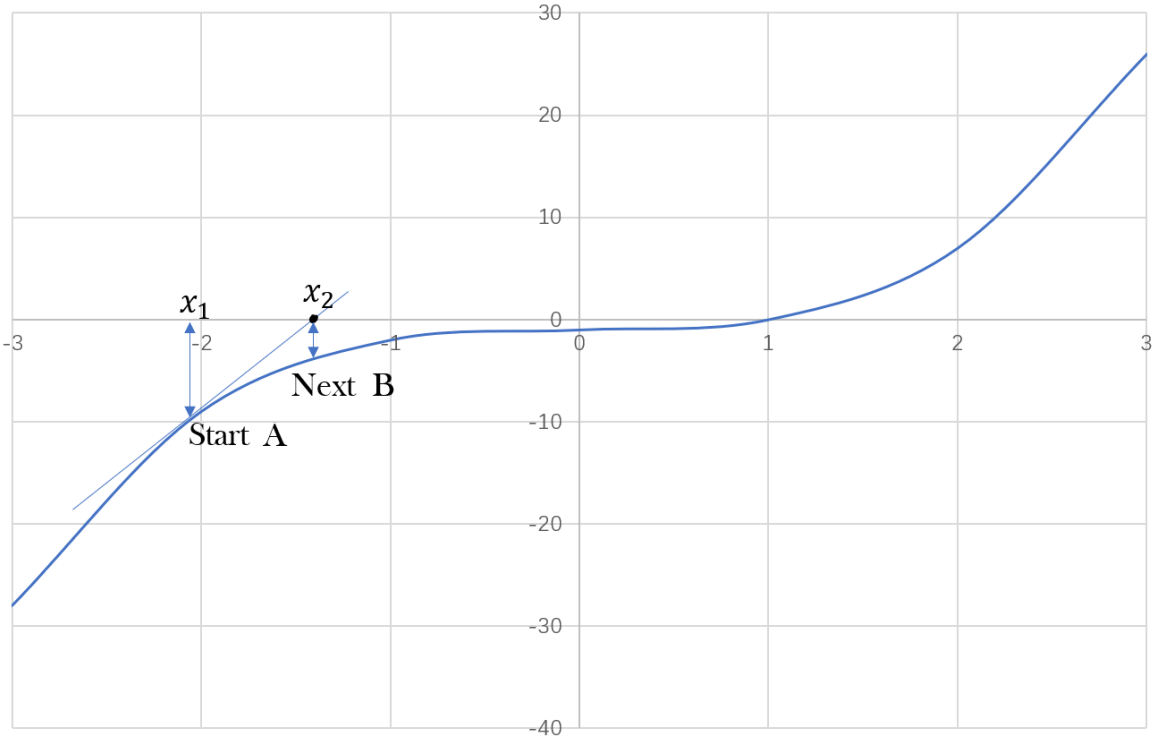
（2）接下来我们求解上述方程的解，即切线的零点，与 x 轴的交点.

只需在切线中令$y = 0$：

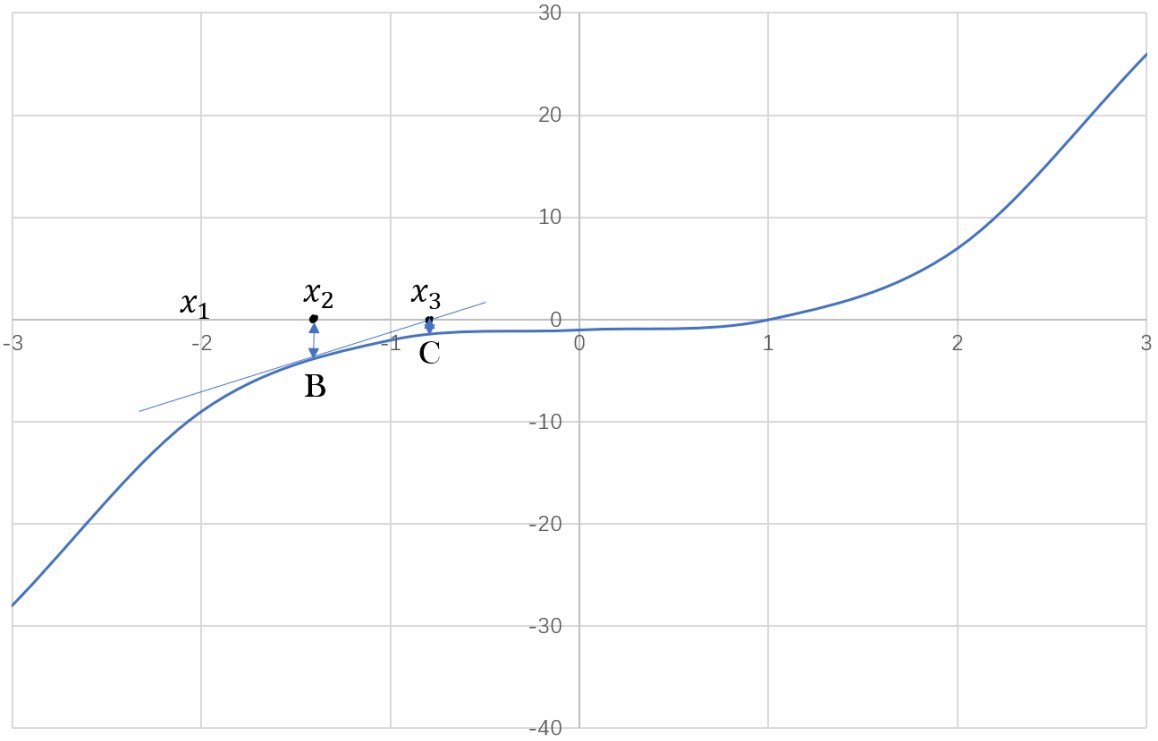
$$0 = f(x_1) + f'(x_1)(x - x_1)$$
$$x - x_1 = \frac{-f(x_1)}{f'(x_1)}$$
$$x = x_1 - \frac{f(x_1)}{f'(x_1)} = x_2$$

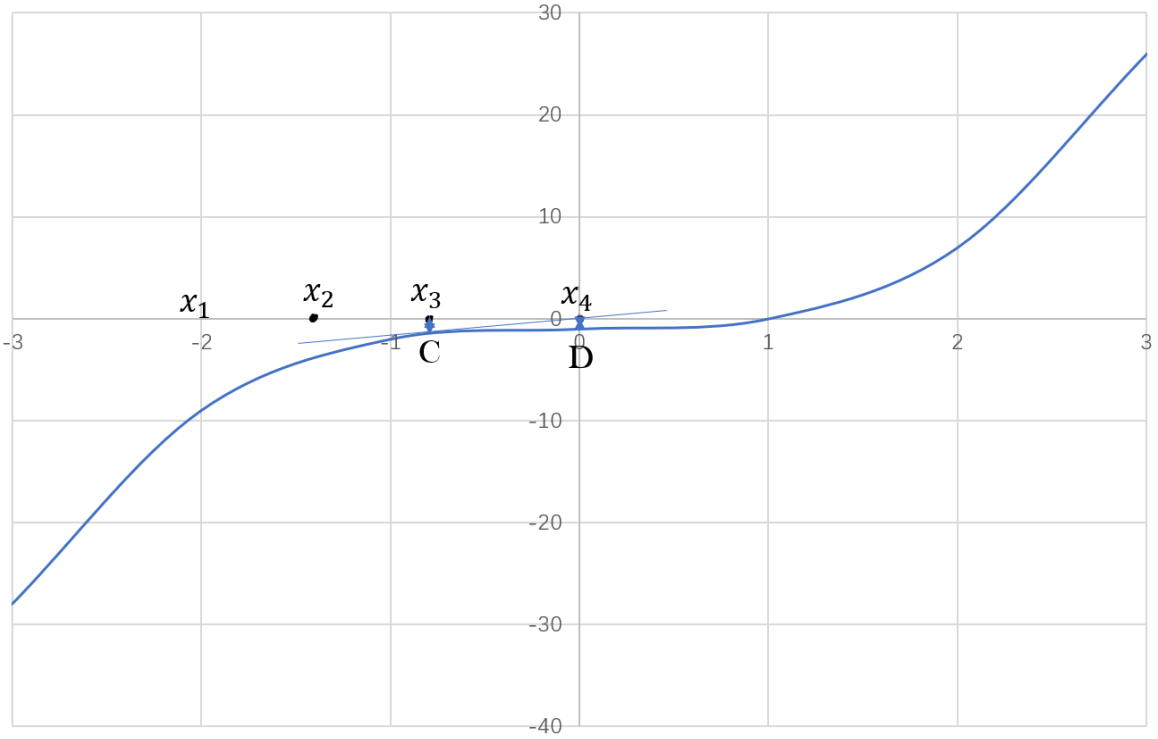
x_2 就是我们选取前往的下一个迭代点，由此我们得到一元函数一阶牛顿法迭代公式：

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



（3）重复迭代步骤直至寻找到或是接近极值点。





最终我们迭代出的 D 点处的切线水平，导数为 0，我们认为已经完成迭代，所需要找的极值点是 D，这就是牛顿法——通过在该点的近似函数的解确定下一个迭代点位置。

二阶牛顿法与一阶牛顿法同理，只是泰勒展开时取前三项，我们在此就不再作图演示（作图不易），只做一些计算上的引导：

（1）计算二阶泰勒展开：

$$f(x) \approx f(x_1) + f'(x_1)(x - x_1) + \frac{1}{2}f''(x_1)(x - x_1)^2$$

$$y = f(x_1) + f'(x_1)(x - x_1) + \frac{1}{2}f''(x_1)(x - x_1)^2$$

（2）求解上述方程的解，即令 $y = 0$ 解得：

$$x = x_1 - \frac{f(x_1)}{f'(x_1)} - \frac{f''(x_1)}{2(f'(x_1))^2}f(x_1)^2 = x_2$$

由此我们得到一元函数二阶牛顿法的迭代公式：

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} - \frac{f''(x_n)}{2(f'(x_n))^2}f(x_n)^2$$

在人工智能领域中我们通常面对着海量的数据，往往都是多元函数，故我们将二阶牛顿法扩展到**多元函数的情况**：

（1）选定初始点 x_0 ，在 x_0 处对函数作二阶泰勒展开：

$$f(x) \approx f(x_0) + \nabla f(x_0)(x - x_0) + \frac{\nabla^2 f(x_0)(x - x_0)^2}{2}$$

对函数两边求梯度：

$$\nabla f(x) \approx \nabla f(x_0) + \nabla^2 f(x_0)(x - x_0)$$

（2） 令梯度$\nabla f(x) = 0$：

$$0 = \nabla f(x_0) + \nabla^2 f(x_0)(x - x_0)$$

这个方程组的解就是我们所需要的下一个迭代点：

$$x = x_0 - (\nabla^2 f(x_0))^{-1}\nabla f(x_0)$$

其中，$\nabla^2 f(x)$ 又被记作 $Hessian$ 矩阵（二阶偏导数构成的方阵）。

通常，我们把 k 处的梯度 $\nabla f(x_k)$ 简记作 g_k ，而 $Hessian$ 矩阵 $\nabla^2 f(x_k)$ 记作 H_k，再添加上我们需要的步长 α ，则迭代公式可以简化为：

$$x_{n+1} = x_n - \alpha H_k^{-1} g_k$$

我们总结一下二阶牛顿法的步骤，对于待优化的函数：

二阶牛顿法
1）选取初始点x_0，确定步长α，确定迭代终止条件ϵ
2）计算当前点的梯度g_k、$Hessian$矩阵H_k

二阶牛顿法
3）计算下一个点的位置$x_0 - \alpha H_k^{-1} g_k$
4）前往下一个点，进行迭代$x \to x_0 - \alpha(\nabla^2 f(x_0))^{-1} \nabla f(x_0)$
5)重复迭代步骤至$f(x_n) - f(x_{n-1}) < \epsilon$或$\nabla^2 f(x_n) \to 0$，则最小值点为$x_n$

则我们可以得到二阶牛顿法的算法伪代码：

二阶牛顿法
$x_0 = ?, k = 0$
while $\|\|\nabla f(x_k)\|\| < \epsilon$ and $k < N$:
$g_k = \nabla f(x_0)$
$H_k = \nabla^2 f(x_0)$
$d_k = -H_k^{-1} g_k$
$x_{k+1} = x_k + \alpha d_k$
$k = k + 1$

我们使用Python复现牛顿法：

```
import numpy

#初始化
#待求函数
def f(x):     #一元待求函数
    return x ** 3 - 2
def df(x):
    return 3 * x ** 2
def d2f(x):
    return 6 * x

k = 0
fx = 0
fx_last = 1
N = 10000  #迭代上限,防止陷入死循环
epsilon = 0.0001  #用于判定是否达到极值
step = 0.01  #步长
x = 100  #初始点
y = 100  #初始点
```

```
#一元函数一阶牛顿法
#导数接近于0时停止
while k < N and abs(df(x)) > epsilon:
    x -= f(x) / df(x)
    fx = f(x)
    k += 1
    print("第%d次: x = %f, f = %f, df = %f" % (k+1, x, fx, df(x)))
```

与梯度下降法不同的是，梯度下降每一步都基本能保证向着目标点方向（尽管步数多），牛顿法步子迈的很大，迭代速度快，但是由于需要计算二阶导数，会增大每次迭代的计算成本。

当然，在多元函数的二阶牛顿法中，大家也一定发现有一个矩阵求逆的过程，自然而然地就会想到，矩阵是否存在不可逆的情况？是的，**当 $Hessian$ 矩阵不可逆时，牛顿法就会失效。**

拟牛顿法（Quasi-Newton Method）

为了解决 $Hessian$ 矩阵不可逆的问题，拟牛顿法诞生了，思路在于，既然 $Hessian$ 矩阵可能无法求逆，那么我们就通过别的方法得到 $Hessian$ 矩阵的逆——构造近似 $Hessian$ 矩阵或其逆矩阵的正定对称矩阵，然后用这个构造的矩阵去进行牛顿法迭代。

因为本人懒，以及拟牛顿法涉及到的原理推导显然不适合于这份入门级的引导教程，所以欲知后事如何，且听下回分解~仅附上相应的两种算法名字，大家可以搜索学习：DFP、BFGS。

iii. 分治

在我们的专业领域里，浩如烟海的数据会让我们无法同时进行优化，分治的思想教导我们将这些高维度的数据分开一维一维地处理，把高维优化问题转化为低维优化问题。举个栗子，在二元的优化问题中寻找最低点，不是选择梯度方向寻找，而是选择固定 y 从 x 方向下降，然后固定 x 从 y 方向下降。

坐标下降法（Coordinate Descent）

对于一个这样的多元优化问题：

$$\min_{x} f(x), x = (x_1, x_2, \cdots, x_n)$$

坐标下降法即是依次选择 $x_1, x_2, \cdots, x_n$ 进行优化，相当于分别进行求解：

$$\min_{x_1} f(x)$$
$$\min_{x_2} f(x)$$
$$\cdots$$
$$\min_{x_n} f(x)$$

事实上坐标下降更像是一种思想，对于坐标下降法就有这样的算法伪代码：

坐标下降法
$x_0 = ?$
while 未收敛:
for $i = 0, 1, \cdots, n:$
求解$\min_{x_i} f(x)$

坐标下降法计算量较小，但是对于光滑（不可导）函数不能有效处理。

SMO算法

II. 解析优化

接下来我们介绍解析优化，顺便我们再复习一下什么是解析优化——能够直接求解的，不用迭代的方法就是解析优化。

i. 等式约束问题

拉格朗日乘数法（Lagrange Multiplier Method）

想必这个方法大家也都不陌生了，这是我们在高等数学中求解函数条件极值的方法，我们简单复习一下。

对于优化问题并伴随限定条件：

$$\min_{x} f(x)$$
$$g(x) = 0$$

拉格朗日乘数法有这样的步骤：

（1）引入拉格朗日乘子 λ 构造拉格朗日乘子函数：

$$L(x, \lambda) = f(x) + \lambda g(x)$$

（2）求解该函数极值点：

即求解该函数偏导数为零的解。

$$\frac{\partial L}{\partial x} = 0$$
$$\frac{\partial L}{\partial \lambda} = 0$$

由上面两式可得极值点 x，这就是等式约束问题的解。

ii. 等式及不等式约束问题

KKT条件

KKT条件是对拉格朗日的推广，用以解决带有等式及不等式约束的问题。

对于优化问题并伴随限定条件：

$$\min_{x} f(x)$$
$$g(x) = 0$$
$$h(x) \leq 0$$

有这样的步骤：

（1）引入KKT乘子 λ 和 μ 构造拉格朗日乘子函数：

$$L(x, \lambda, \mu) = f(x) + \lambda g(x) + \mu h(x)$$

（2）求解该函数极值点：

即求解该函数偏导数为零的解。

$$\frac{\partial L}{\partial x} = 0$$
$$\frac{\partial L}{\partial \lambda} = 0$$
$$\frac{\partial L}{\partial \mu} = 0$$
$$\mu h(x) = 0$$

由上面两式可得极值点 x，这就是等式及不等式约束问题的解。

KKT条件给出了等式及不等式约束优化问题的一阶必要条件。

2. 多目标优化

多目标优化即是既要又要，同时使得多个目标函数取到理想的值。当然大家也可以想象得到，多目标优化往往是不可能真的同时使多个函数取到最值的，只能是尽量使多个函数的取值满足我们最低限度的需求，也就是，大多数多目标优化不存在最优解一说。

我们依然按照描述优化问题的方式描述多目标优化问题：

$$\underset{x}{min}\ (f_1(x), f_2(x), \cdots, f_n(x))$$

我们的使命就是求解合适的 x 使得 $f_1(x)$ 、$f_2(x)$ 、···、$f_n(x)$均能尽可能地取到最小值。

标量化法

标量化法的思想其实与拉格朗日乘子法有异曲同工之妙——构造新的函数把条件包含进去，然后求解新函数的极值。

对于多目标优化问题：

$$\underset{x}{min}\ (f_1(x), f_2(x), \cdots, f_n(x))$$

标量化法将其转化为求解：

$$\underset{x}{min}\ g(f_1(x), f_2(x), \cdots, f_n(x), \theta)$$

g 是我们自己设计的标量化函数伴随参数 θ。

最简单的标量化方法就是线性标量化，即将目标函数通过线性加权构造新函数：

$$\underset{x}{min}\ (w_1 f_1(x) + w_2 f_2(x) + \cdots + w_n f_n(x))$$

ϵ 约束法

ϵ 约束法的思想是拉格朗日乘子法的延续——只留下一个目标函数，将其他目标函数转变成不等式约束，然后通过KKT条件求解。

$$\begin{gathered}\underset{x}{min}\ f_1(x)\\ f_2(x) \le \epsilon\\ f_3(x) \le \epsilon\\ \cdots\\ f_n(x) \le \epsilon\end{gathered}$$

3. 特殊情况的优化

凸优化